



GitHub Trending Top 10

每日热门开源项目深度解读

2026-10-10

今日榜单

1	morluto/rea	TypeScript	Reverse engineer anything with agents, from app behavior down to native binaries...	5天
2	boykopovar/AnyPS5	C++	Tool for automatic PS5 executables porting to Linux and Windows	6天
3	storytold/artcraft	Rust	ArtCraft is an intentional crafting engine for artists, designers, and filmmaker...	3天
4	cathrynlavery/diagram-design	HTML	Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, ...	4天
5	mksglu/context-mode	TypeScript	Context window optimization for AI coding agents. Sandboxes tool output (98% red...	NEW
6	mattpocock/skills	Shell	Skills for Real Engineers. Straight from my .agents directory.	5天
7	flutter/flutter	Dart	Flutter makes it easy and fast to build beautiful apps for mobile and beyond	NEW
8	tensorflow/tensorflow	C++	An Open Source Machine Learning Framework for Everyone	NEW
9	hugohe3/ppt-master	Python	AI turns documents or topics into real, native PowerPoint decks—with native shap...	NEW
10	pytorch/pytorch	Python	Tensors and Dynamic neural networks in Python with strong GPU acceleration	NEW

#1

TypeScript

连续 5 天

rea

Reverse engineer anything with agents, from app behavior down to native binaries.

Stars **64,596** Forks **13,318** Today **+25,784**

<https://github.com/morluto/rea>

项目简介：REA 是一个基于 MCP（Model Context Protocol）的逆向工程工具集，让 AI Agent 能够分析原生二进制、JavaScript/Electron 应用、.NET 程序集和网站的行为。它解决的核心问题是：当你看到一个 App 里想要的功能，却没有源码时，可以借助 Agent 去"看懂"它到底怎么实现的，并据此在自己的项目里复刻。

核心功能：

- **多目标逆向分析：**覆盖原生二进制（对接 Hopper/Ghidra/IDA）、JS/Electron 应用、.NET 程序集以及网站，统一在一个工具链下。
- **Agent 驱动的工作流：**通过 MCP 把分析能力接入 Claude Code、Cursor、Codex、Gemini CLI 等主流 Agent，用自然语言下达分析指令。
- **证据化结论：**每个分析结果都附带证据和局限性说明，而非黑盒输出，方便验证和追溯。
- **终端直用：**不依赖 Agent 也能从命令行直接调用（如 `analyze-javascript-application`），支持 JSON 输出便于集成。
- **本地化运行：**分析在本地执行，代码和二进制不外传，兼顾隐私与合规。

适用场景：产品经理或工程师看到竞品/第三方 App 的某个功能想借鉴时；安全研究人员做二进制或应用行为分析时；开发者在没有源码的情况下需要理解 Electron/JS 应用的模块结构与边界时。简言之，任何"想搞懂某个软件怎么做到的"场景都适用。

技术亮点：

- 以 MCP 作为 Agent 与逆向工具之间的标准协议层，把原本分散的 Hopper、Ghidra、IDA 等工具统一抽象，降低了 Agent 集成的复杂度。
- 强调"证据 + 局限"的输出范式，这在 AI 辅助分析领域是相对克制的设计，有助于避免模型幻觉被当作事实。
- 静态 JS 分析无需原生引擎即可运行，降低了入门门槛；原生分析则复用用户已有的商业/开源工具，避免重复造轮子。

上手建议：先确保本机有 Node.js 和 npm，运行 `npx rea-agents setup`，按提示选择你的 Agent 并确认配置变更（会自动备份原配置），重启 Agent 后即可用自然语言提问。想先体验可从纯 JS/Electron 目标入手（无需 Hopper/Ghidra），熟悉后再配置原生分析引擎。若要贡献代码，建议先阅读 `docs/installation.md` 和 `docs/mcp-contracts.md`，理解 MCP 工具契约和 Agent 工作流的设计约定，再针对具体分析器或 Agent 适配做扩展。

#2

C++

连续 6 天

AnyPS5

Tool for automatic PS5 executables porting to Linux and Windows

Stars **25,269** Forks **2,101** Today **+5,831**

<https://github.com/boykopovar/AnyPS5>

项目简介：AnyPS5 是一个将 PS5 可执行文件自动移植到 Linux 和 Windows 平台的开源工具。它通过“重链接器”将 PS5 二进制转换为目标系统的原生格式，并配合系统 PRX 库实现动态链接，从而让 PS5 游戏无需模拟器即可在 PC 上原生运行。

核心功能：- **重链接器 (Relinker)**：将 PS5 可执行文件转换为 Linux/Windows 原生可执行格式，而非依赖模拟层。- **系统 PRX 库实现**：提供适用于动态链接的 PS5 系统库函数实现，逐步覆盖已知的系统 API。- **着色器重编译器**：将 PS5 着色器成功重编译为 SPIR-V，并通过 Spirv-Tools 验证。- **输入映射支持**：兼容 SDL 手柄（含摇杆和扳机），并支持通过 `anyps5-input.ini` 配置键鼠操作。- **兼容性追踪**：提供已验证游戏列表和进度地图，透明展示项目覆盖范围。

适用场景：主要面向游戏保存与互操作性研究者、希望在没有 PS5 硬件的情况下运行特定游戏的玩家，以及对二进制转换和系统库重实现感兴趣的系统级开发者。目前已有《Dreaming Sarah》在 GTX 1050 Ti / i5-7500 上稳定 60fps 运行的验证案例。

技术亮点：项目最大的特点是**不做模拟、不启动独立运行时进程**，而是直接进行静态重链接和原生动态链接，这在架构上比传统模拟器更轻量、性能更接近原生。同时，项目对异常状态采取严格的 `std::runtime_error` 抛出策略并终止进程，体现出对正确性的高要求；着色器重编译到 SPIR-V 的路径也已打通并经过工具验证。

上手建议：普通用户应先阅读 `docs/user/USAGE.md` 了解使用流程，并查看 `docs/user/COMPATIBILITY.md` 确认目标游戏是否受支持。开发者贡献可从 `docs/dev/BUILD.md` 构建项目、`docs/dev/ARCHITECTURE.md` 理解架构入手，并遵循 `docs/dev/CONVENTIONS.md` 的代码风格；若想参与库函数实现，可重点关注 `core/libs/prx` 目录。

#3

Rust

连续 3 天

artcraft

ArtCraft is an intentional crafting engine for artists, designers, and filmmakers

Stars **13,380** Forks **2,070** Today **+3,217**

<https://github.com/storytold/artcraft>

项目简介

ArtCraft 是一个面向艺术家、设计师和电影制作人的「AI 创作 IDE」，定位为「艺术家的 IDE」。它试图解决当前 AI 图像/视频生成工具的核心痛点：纯靠文字提示词（prompting）难以精确控制构图、角色一致性和镜头位置。ArtCraft 的思路是让创作者先在 2D/3D 空间中搭建场景，再调用 AI 模型生成，把「提示词抽奖」变成可重复、可精确控制的「crafting（手工艺创作）」。

核心功能

- **Image to Location**：把虚拟角色放进一致的环境中，在同一「房间」里规划多个镜头，保证场景连贯性。
- **2D/3D 图像合成**：支持图层化合成、背景移除、绘图工具，以及在 3D 空间中分层布置背景、前景和道具，构建有纵深的构图。
- **Image to 3D Mesh**：将一张图片转成可旋转、可定位的 3D 物体，用于精确取景。
- **角色摆姿与镜头预设**：在生成最终画面之前，先摆好角色姿势、设定摄像机位置，实现「先布景、后生成」。
- **Kitbashing 场景搭建**：组合 3D 资产套件来控制机位角度、物体摆放和景深。

适用场景

主要面向需要**精确控制画面**的视觉创作者：分镜师、概念设计师、独立电影人、广告/短视频创作者，以及需要保持角色与场景一致性的连载内容制作者。典型用法是：先搭好场景和角色站位，规划多个镜头，再让 AI 在既定构图中生成图像或视频，从而避免反复「抽卡」式生成。

技术亮点

- 用 **Rust** 编写，说明项目对性能、跨平台桌面体验和内存安全有较高要求，符合其「IDE 级」桌面应用的定位。

- 架构上把「场景搭建（2D/3D 合成、摆姿、机位）」与「模型生成」解耦，用户可自选适配的 AI 模型，体现了**模型无关（model-agnostic）**的设计哲学。
- 将传统 3D 内容创作工具的工作流（合成、kitbash、镜头调度）与生成式 AI 结合，是「可控生成」这一方向的工程化尝试。

上手建议

普通用户可直接从 getartcraft.com 下载安装包，或从 GitHub Releases 获取版本，并先看官方两分钟演示视频了解工作流。开发者若想参与贡献，README 中提供了 `_docs/dev_setup.md` 的从源码构建指南，建议先按此搭建 Rust 开发环境；同时可加入其 Discord 社区跟进讨论。鉴于项目星标增长极快（今日 +3217），建议关注 issue 区和发布节奏，从文档、示例或小 bug 入手较为稳妥。

#4

HTML

连续 4 天

diagram-design

Editorial diagram design for Claude Code, Codex, GitHub Copilot, Factory Droid, and Pi. 44 diagram types. Self-contained HTML + SVG. No shadows. No Mermaid slop.

Stars **48,601** Forks **3,070** Today **+1,189**

<https://github.com/cathrynlavery/diagram-design>

项目简介：这是一个面向 AI 编程助手（Claude Code、Codex、Copilot、Cursor 等）的「Agent Skill」，让 AI 在对话中用自然语言直接生成编辑级品质的图表。它解决的是 Mermaid 等工具生成图表样式粗糙、千篇一律的痛点——用自包含的 HTML+SVG 输出可直接双击打开的成品图。

核心功能：

- 覆盖 44 种图表类型（架构图、象限图、时序图等），由 AI 自动选择合适类型并说明规划。
- 一键「品牌化」：通过「onboard diagram-design to yoursite.com」抓取你网站的配色和字体，后续所有图表自动匹配品牌。
- 输出为单个自包含 .html 文件，无需构建工具或依赖，双击即可查看。
- 提供 /export-diagram、/import-mermaid、/profile、/doctor 等斜杠命令（原生包）。
- 内置设计规则（无阴影、编辑级排版），从源头避免「Mermaid 味」。

适用场景：适合需要快速产出演示级、文档级图表的开发者、技术写作者和产品团队，尤其是做架构说明、项目优先级象限、API 调用时序等场景。任何已在用 AI 编程助手、又厌倦了手工调图或 Mermaid 默认样式的人都适用。

技术亮点：以 Agent Skill 形式跨宿主分发（通过 `npx skills add` 或各家 marketplace），一份技能装入多个 AI 工具根目录，还支持 symlink 统一更新。用纯 HTML+SVG 而非图片或绘图库，保证了零依赖、可移植和矢量清晰度，同时把「设计规范」编码进技能本身，让 AI 输出天然带审美约束。

上手建议：最简单是运行 `npx skills add cathrynlavery/diagram-design`，或在 Claude Code 中通过 `/plugin marketplace add` 安装并开启自动更新。装好后直接用自然语言提需求（如「画一个带 Redis 缓存的应用架构图」）即可。想贡献可从仓库的 `skills/diagram-design/SKILL.md` 和 `references/` 入手，理解图表类型定义与设计规则的组织方式。

#5

TypeScript

NEW

context-mode

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

Stars **26,113** Forks **1,881** Today **+178**

<https://github.com/mksglu/context-mode>

项目简介：Context Mode 是一个面向 AI 编程代理的 MCP 服务器，专门解决上下文窗口被工具输出快速填满的痛点。它通过沙箱化工具输出、持久化会话记忆和跨平台路由，让代理在长时间编码任务中保持“记得住、接得上”。

核心功能：一是沙箱化工具输出，把 Playwright 快照、GitHub issue、日志等原始数据挡在上下文之外，宣称可减少 98% 的上下文占用；二是会话连续性，用 SQLite + FTS5 索引文件编辑、git 操作、任务和错误，压缩后只通过 BM25 检索相关片段；三是“Think in Code”，引导模型用代码而非冗长自然语言来思考和表达；四是跨 17 个平台的路由，通过 MCP + hooks 统一接入。

适用场景：主要面向使用 Cursor、Claude Code 等 AI 编程代理的开发者，尤其是需要长时间、多步骤、频繁调用工具（如浏览器快照、GitHub API、日志分析）的团队。当代理频繁因上下文压缩而“失忆”、忘记正在编辑的文件或任务进度时，这个项目价值最明显。

技术亮点：用 SQLite + FTS5 做事件索引和 BM25 检索，而不是把历史数据重新塞回上下文，这是比较聪明的“外部记忆”设计；同时强调“不 --continue 就立即删除上一会话数据”，在隐私和状态干净之间做了明确取舍；沙箱化工具输出则从源头控制上下文膨胀，而非事后压缩。

上手建议：先从 npm 安装 context-mode，在支持的 MCP 客户端里配置为服务器，观察一个典型编码会话的上下文占用变化。想贡献的话，优先看 README 中“Think in Code”和路由部分，以及 stats.json 相关的用量统计逻辑，那里最能体现项目核心假设和可扩展点。注意 License 是 ELv2，商用前需确认合规。

#6

Shell

连续 5 天

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **283,910** Forks **23,782** Today **+1,737**

<https://github.com/mattpocock/skills>

项目简介：这是 TypeScript 教育者 Matt Pocock 开源的 AI 编程助手技能集（Agent Skills），从他自己日常使用的 .agents 目录中提取而来。它针对当前 AI 编码代理（如 Claude Code、Codex、Copilot）常见的两大失败模式——需求理解偏差和输出啰嗦——提供轻量、可组合的解决方案，强调“真正的工程”而非“凭感觉编码”（vibe coding）。

核心功能：最突出的是 /grill-me 和 /grill-with-docs 两个“拷问”技能，让代理在动手前反复追问你的真实意图，从而对齐需求；其次是 /triage 配合自定义标签做工单分类；还有 /setup-matt-pocock-skills 一次性初始化技能，配置 issue tracker、标签和文档路径。整体技能按 engineering 和 productivity 两大类组织，可自由挑选组合。

适用场景：适合日常使用 Claude Code、Codex、GitHub Copilot、Gemini CLI、Cursor 等编码代理的开发者，尤其是希望把 AI 纳入严肃工程流程、而非只做玩具项目的人。典型场景是：开始一个改动前先让代理“拷问”自己、把模糊需求逼成清晰规格；或是在多代理环境里统一 workflow。

技术亮点：项目本身几乎全是 Shell 和 Markdown，没有复杂依赖，核心价值在于“提示词即技能”的设计哲学。它刻意保持小而可改，兼容任意模型，并针对不同代理提供插件市场、npx skills 等多种安装路径。README 明确反对 GSD、BMAD、Spec-Kit 这类“接管流程”的重型框架，认为它们剥夺了开发者控制权、让流程内的 bug 难以定位——这种“反框架”的克制立场本身就是亮点。

上手建议：最快的方式是按 README 的 30 秒安装步骤，选一个你常用的代理执行对应命令，然后运行一次 /setup-matt-pocock-skills 完成仓库级配置。新手建议从 /grill-me 开始体验，感受“先对齐再动手”的差异；想深入或贡献，可阅读 skills/engineering 与 skills/productivity 下的 SKILL.md，直接 fork 改成自己的版本——作者本人也鼓励“hack around with them”。

#7

Dart

NEW

flutter

Flutter makes it easy and fast to build beautiful apps for mobile and beyond

Stars **179,344** Forks **33,546** Today **+39**

<https://github.com/flutter/flutter>

项目简介：Flutter 是 Google 推出的开源跨平台 UI 框架，用一套 Dart 代码即可构建移动端、Web 和桌面端应用。它解决的核心问题是：让开发者不必为每个平台单独写一套 UI 代码，同时还能保持原生级别的渲染性能和视觉表现力。

核心功能：一是跨平台统一开发，覆盖 iOS、Android、Web、Windows、macOS、Linux 甚至嵌入式平台；二是丰富的 Widget 体系，内置 Material 和 Cupertino 两套设计语言，支持像素级自定义；三是热重载（Hot Reload），改代码后即时看到效果且不丢失应用状态；四是强大的图形合成能力，可自由叠加动画、视频、文字和控件；五是完善的包生态和原生互操作能力（FFI、Platform Channels）。

适用场景：适合需要快速迭代、多端一致体验的产品团队，尤其是创业公司或中小团队用一套人力覆盖多平台；也适合需要高度定制 UI 的设计驱动型应用，以及将 Flutter 作为嵌入式 UI 工具链的平台开发者。

技术亮点：底层依赖 Skia 和自研的 Impeller 渲染引擎，绕过平台原生控件体系直接做硬件加速的 2D 绘制，因此能做到“无 jank”的流畅度；Dart 语言支持 AOT 编译为 ARM 机器码，也能编译为 JavaScript/WebAssembly，兼顾性能与跨端；分层架构让开发者可以控制屏幕上的每一个像素。

上手建议：使用方从官网安装 SDK 后跟一遍官方教程即可快速入门，重点理解 Widget 树和状态管理；贡献方建议先阅读 CONTRIBUTING.md 和 docs/README.md，从标记为“good first issue”的问题入手，并加入 Discord 社区与维护者沟通。

#8

C++

NEW

tensorflow

An Open Source Machine Learning Framework for Everyone

Stars **200,628** Forks **78,907** Today **+24**

<https://github.com/tensorflow/tensorflow>

项目简介：TensorFlow 是由 Google Brain 团队开发并开源的端到端机器学习平台，旨在为研究者和开发者提供从模型研究到生产部署的完整工具链。它解决了机器学习开发中框架碎片化、跨平台部署困难以及从实验到上线流程割裂的问题。

核心功能：提供稳定的 Python 和 C++ API，支持快速构建和训练神经网络；内置完整的工具、库和社区资源生态，覆盖数据处理、模型训练、评估与部署全流程；支持 CPU、CUDA GPU、DirectX 及 macOS Metal 等多种硬件加速后端；提供 tensorflow-cpu 轻量包和 nightly 构建版本，满足不同安装与测试需求。

适用场景：机器学习研究者可用它推进前沿算法实验，应用开发者可借助它快速构建和部署 ML 驱动的产品功能，企业团队可用于生产环境中的大规模模型训练与推理服务。教育场景中，其简洁的 API 也适合教学演示和入门实践。

技术亮点：核心以 C++ 实现，兼顾性能与跨语言扩展能力；同时提供稳定的 Python/C++ API 和非保证向后兼容的其他语言 API，平衡了易用性与灵活性；项目采用 OpenSSF Scorecard、OSS-Fuzz 模糊测试、CII 最佳实践等多项安全与质量保障机制，工程成熟度极高。

上手建议：初学者可直接通过 `pip install tensorflow` 安装并运行官方教程中的示例代码快速体验；需要 GPU 支持或自定义构建的用户应参考官方安装指南中的 GPU、Docker 或源码编译章节。有意贡献者应先阅读 CONTRIBUTING.md 和 CODE_OF_CONDUCT.md，并通过 GitHub Issues 跟踪缺陷、通过 TensorFlow Forum 和 Stack Overflow 进行讨论与提问。

#9

Python

NEW

ppt-master

AI turns documents or topics into real, native PowerPoint decks—with native shapes, transitions and animations, data-backed charts and tables on demand, audio narration from speaker notes, and support for your own .pptx templates. • by Hugo He

Stars **59,130** Forks **4,664** Today **+372**

<https://github.com/hugohe3/ppt-master>

项目简介：PPT Master 是一个用 AI 把任意文档或主题直接转换成「原生 PowerPoint 文件」的开源工具——不是导出图片或网页，而是生成可在 PowerPoint 中继续编辑的 .pptx。它解决的是「AI 生成 PPT 但成品不可编辑、样式呆板」这一常见痛点，由 Hugo He 开发，Python 编写，目前已有近 6 万星标。

核心功能：一是把 PDF、DOCX、网页等源材料交给大模型理解后自动提炼要点、组织结构并生成幻灯片；二是输出的是带原生形状、转场和动画的 PPTX，而非静态图片；三是支持按需生成数据驱动的图表和表格；四是能把演讲者备注转成音频旁白；五是支持导入用户自己的 .pptx 模板，保持品牌样式统一。

适用场景：适合需要频繁做汇报、路演、教学课件的职场人士、创业者和教师，尤其是手头已有大量文档（研报、论文、需求文档）希望快速转成演示稿的场景；也适合希望把 AI 生成能力集成到自己产品中的开发者。

技术亮点：关键在于「原生 PPTX 生成」——直接操作 OOXML 结构而非截图或 HTML 渲染，保证文件在 PowerPoint 中完全可编辑；同时借助赞助商 Kimi K3 等具备原生视觉和百万级上下文窗口的模型，能一次性「读懂」长文档并把握叙事结构，这是它区别于普通文本摘要工具的核心。

上手建议：从 README 的安装与快速开始部分入手，先配置一个支持视觉的 LLM API（项目赞助商如 Kimi、PackyCode 等提供了折扣入口），用一份自己的 PDF 或 DOCX 跑通端到端流程；想贡献代码可从 issues 中标记的 good first issue 或模板兼容性、导出格式等方向切入。

#10

Python

NEW

pytorch

Tensors and Dynamic neural networks in Python with strong GPU acceleration

Stars **104,060** Forks **31,908** Today **+81**

<https://github.com/pytorch/pytorch>

项目简介：PyTorch 是 Meta（原 Facebook）开源的一款深度学习框架，核心目标是为 Python 开发者提供兼具灵活性与高性能的数值计算和神经网络构建能力。它解决了传统静态图框架（如早期 TensorFlow）调试困难、灵活性不足的问题，让研究者可以用纯 Python 的方式定义和修改模型。

核心功能：一是 GPU 加速的张量计算库（torch），用法类似 NumPy 但支持 CUDA/ROCm/Intel GPU；二是基于磁带式自动微分（torch.autograd）的动态计算图，支持任意可微操作的自动求导；三是 torch.nn 神经网络模块，与 autograd 深度集成，提供层、损失函数等构建块；四是 torch multiprocessing 支持跨进程共享张量内存，方便数据加载和 Hogwild 训练；五是 torch.utils 提供 DataLoader 等实用工具。

适用场景：主要面向深度学习研究人员和工程师，用于学术论文实验、模型原型快速迭代、生产环境模型训练与部署。任何需要 GPU 加速数值计算或构建动态神经网络的项目都可以使用，从计算机视觉、NLP 到强化学习均有广泛应用。

技术亮点：最值得关注的是其"define-by-run"的动态图机制——计算图在每次前向传播时即时构建，使得调试、条件分支、变长输入等操作自然流畅，无需预先定义静态图结构。此外，"Python First"的设计理念让框架与 Python 生态（NumPy、SciPy、Cython）无缝互操作，降低了学习和扩展成本。

上手建议：初学者可从官网的 60 分钟入门教程和 "Learn the Basics" 系列开始，配合 pip/conda 安装二进制包即可快速跑通示例。若想贡献代码，建议先阅读 CONTRIBUTING.md，从修复文档或标注 "good first issue" 的简单 bug 入手，并关注 hud.pytorch.org 了解 CI 健康状况。

数据来源: github.com/trending

报告日期: 2026-10-10

由 DeepSeek AI 提供智能分析 | 自动生成